



VOXA STREAM

Resumo completo do projeto e funcionamento técnico

Versão analisada: 0.7.4

Plataforma: Windows x86-64

Arquitetura, operação, segurança e manutenção

Baseado no repositório em 28 de setembro de 2026

Sumário

Sumário	2
1. Visão geral	4
Resumo executivo	4
O que o produto faz	4
Fora do escopo atual	4
2. Arquitetura do sistema	5
Separação de responsabilidades	5
Topologia de execução	5
Camadas do repositório	5
3. Fluxo de conexão e aprovação	6
Sequência completa	6
Estados do motor	6
Multiponto	6
4. Pipeline de vídeo nativo	7
Captura no host	7
Conversão e encode	7
Perfis adaptativos	7
Decodificação e apresentação	7
5. Áudio da transmissão	8
Origem e isolamento	8
Formato e sincronização	8
6. Transporte UDP e redes ruins	9
Protocolo de datagramas	9
Perda, FEC e keyframes	9
Fila, pacing e CPU	9
NAT traversal e relay	9
7. Segurança e modelo de confiança	10
Proteções implementadas	10
Identidade persistente	10
Segurança do desktop e da entrega	10
8. Signaling, relay e infraestrutura	11
Servidor de matchmaking	11
Implantação gratuita	11
Variáveis operacionais	11

9. Interface e ciclo de vida	12
Painel principal	12
Janela de streaming	12
Bandeja e memória	12
Atualizador	12
10. Diagnóstico e recuperação	13
Telemetria	13
Gravador circular e exportação	13
Recuperação automática	13
11. Estrutura e dimensão do código	14
Catálogo dos módulos nativos	14
Dependências centrais	14
12. Testes, CI e qualidade	15
Cobertura automatizada	15
Integração contínua	15
Pipeline de release	15
Estado da versão 0.7.4	15
13. Como usar e operar	16
Para hospedar	16
Para assistir	16
Solução de problemas	16
14. Desenvolvimento e implantação	17
Ambiente local	17
Signaling no Render	17
Relay na Oracle	17
Versionamento	17
15. Limitações e riscos conhecidos	18
Critério recomendado de prontidão	18
16. Próximos passos recomendados	19
Prioridade 1 - validar o produto	19
Prioridade 2 - resiliência operacional	19
Prioridade 3 - manutenção	19
17. Glossário	20
18. Referências internas e conclusão	21
Conclusão	21

1. Visão geral

O Voxa Stream é um aplicativo desktop de transmissão remota de vídeo e áudio para Windows. Seu objetivo é enviar a tela de um computador host a até quatro espectadores com baixa latência. Captura, codificação, transporte, decodificação e exibição são nativos. O painel React serve para configurar e acompanhar a sessão; o conteúdo não passa por elementos HTML de mídia.

O produto atual nasceu de uma mudança de direção: chat, voz por microfone, perfis, anexos, Supabase e mídia WebRTC foram removidos. O foco da linha 0.7 é streaming P2P por UDP nativo, com relay cego quando a rota direta não funciona.

Resumo executivo

Área	Implementação atual
Painel	Tauri 2, React 19, TypeScript e Vite
Motor	Rust, Tokio, APIs Win32, DXGI, D3D11, Media Foundation e WASAPI
Vídeo	Captura GPU, conversão NV12 e codecs AV1, H.265 e H.264 por hardware
Áudio	Loopback do sistema ou de processo, Opus 48 kHz estéreo
Rede	UDP nativo, STUN, hole punching, relay cego e controle de congestionamento
Segurança	Argon2id, SPAKE2 P-256, X25519, ChaCha20-Poly1305 e antirreplay
Capacidade	Um host e até quatro espectadores com encoder e métricas independentes
Distribuição	EXE NSIS e MSI assinados, com atualização pelo GitHub Releases

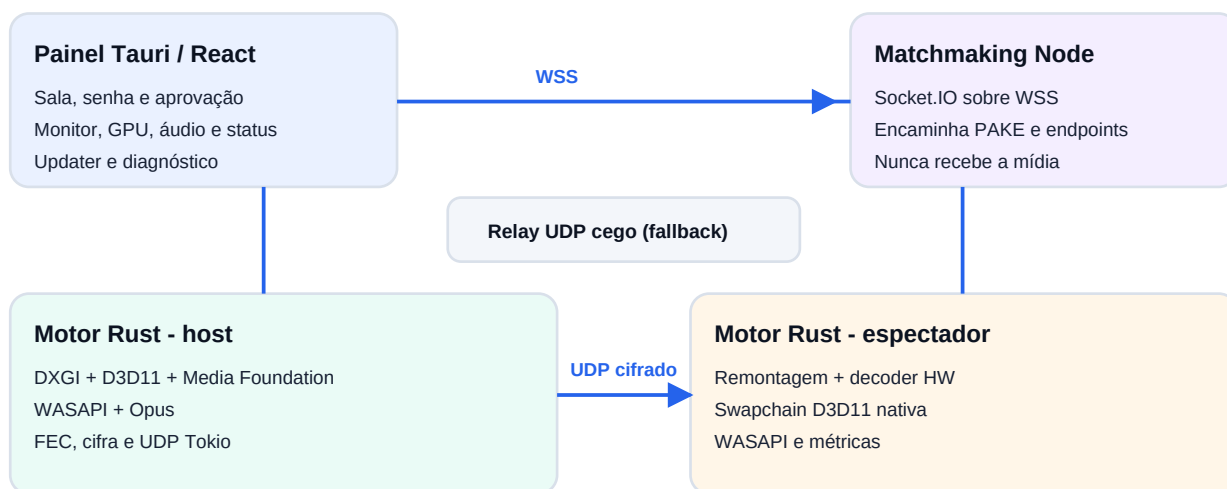
O que o produto faz

- Hospeda tela e áudio de um PC Windows.
- Conecta como espectador e exibe a transmissão numa janela nativa separada.
- Permite escolher monitor, GPU, fonte de áudio, GPU de decodificação e visibilidade do cursor.
- Adapta bitrate, resolução, FPS e bitrate de áudio conforme a rede.
- Exige aprovação de cada espectador por código E2E antes de liberar mídia.
- Reconecta após mudanças de rede e recupera captura, encoder, decoder e dispositivo gráfico.
- Abre uma segunda instância para hospedar e assistir simultaneamente.

Fora do escopo atual

- Chat de texto, voz por microfone, perfis, imagens, anexos e presença social.
- Banco Supabase e autenticação de contas na nuvem.
- Transporte de mídia por WebRTC ou TURN.
- Controle remoto de teclado, mouse ou gamepad.
- Suporte oficial a Linux, macOS, Android ou navegador.

2. Arquitetura do sistema



Separação de responsabilidades

Componente	Responsabilidade	O que não faz
React/Tauri	Formulário, IPC, aprovação, status, updater e diagnóstico	Não captura nem reproduz mídia
Motor Rust	Captura, codecs, áudio, UDP, criptografia, telemetria e janela nativa	Não mantém contas ou histórico em cloud
Signaling Node	Matchmaking e encaminhamento de mensagens públicas	Não recebe senha, chave privada ou mídia
Relay UDP	Encaminha pacotes cifrados quando a rota direta falha	Não descriptografa frames
GitHub Actions	Audita, testa, empacota, assina e publica	Não promove release com checks falhos

Topologia de execução

Cada computador executa o mesmo aplicativo. O papel escolhido no painel é host ou espectador. O host abre um socket UDP, descobre sua rota por STUN, prepara captura e encoders e aguarda aprovação. O espectador abre seu socket, detecta decoders de hardware e cria a janela nativa após o pareamento.

O signaling fica fora do caminho da mídia. Depois que as rotas são negociadas, o tráfego pesado circula diretamente entre os computadores ou pelo relay UDP cego.

Camadas do repositório

Camada	Diretório	Papel
Interface	src	Painel React, IPC e cliente Socket.IO
Aplicativo nativo	src-tauri/src	Tauri, ciclo de vida e motor multimídia
Núcleo puro	src-tauri/native-core	Protocolo, cifra, STUN, FEC e congestionamento
Servidor	server	Matchmaking HTTP/WSS e relay UDP
Automação	scripts e .github	Testes, auditoria, build e release

3. Fluxo de conexão e aprovação

Sequência completa

- 1 O usuário informa código da sala e uma senha de 12 a 128 caracteres.
- 2 O Rust deriva um segredo de 32 bytes com Argon2id, 64 MiB, três iterações e salt vinculado à sala. A operação roda fora da thread da interface.
- 3 Host e espectador entram na sala pelo Socket.IO sobre HTTPS/WSS e anunciam apenas função, capacidades e material público.
- 4 Os pares executam SPAKE2 P-256 com confirmação mútua. O servidor somente encaminha as mensagens públicas.
- 5 A identidade X25519 deriva chaves de mídia independentes e um código E2E de seis dígitos.
- 6 O host compara o código e aprova o computador, ou aceita um par previamente confiável no Gerenciador de Credenciais do Windows.
- 7 Somente depois da aprovação o signaling libera endpoints e credenciais individuais de relay.
- 8 Os clientes tentam hole punching direto e relay em paralelo. A primeira rota válida e autenticada vence.
- 9 Configuração de codec, dimensões e FPS é repetida no túnel. O espectador pede um keyframe e começa a apresentar os frames.

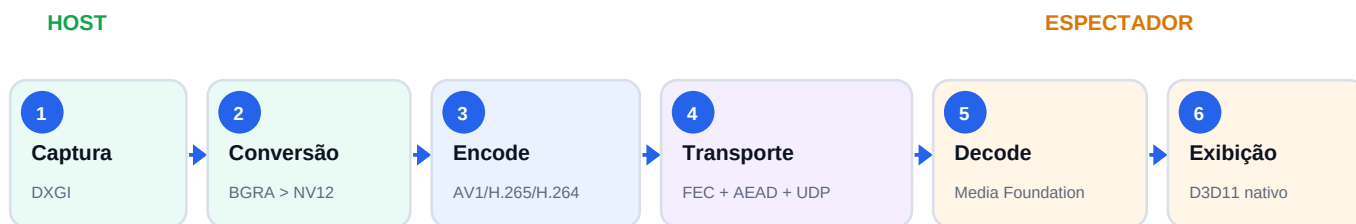
Estados do motor

Estado	Significado
idle / stopped	Nenhuma sessão ativa
binding	Socket, rota e identidade sendo preparados
waiting	Aguardando par ou aprovação
punching	Testando rota direta e relay
connected / decoding / streaming	Rota ativa e pipeline funcionando
recovering	Reconstruindo rede, captura, codec ou dispositivo
failed	Falha que exige diagnóstico ou nova tentativa

Multiponto

O matchmaking aceita um host e até quatro espectadores. Cada espectador recebe transporte, chave, relay, bitrate, perfil de vídeo, encoder e métricas próprios. A saída de um espectador não encerra os demais, e quem entra depois recebe configuração repetida e um novo IDR.

4. Pipeline de vídeo nativo



Texturas permanecem na GPU; a mídia não passa por uma tag HTML video.

Captura no host

O módulo capture.rs enumera adaptadores e saídas DXGI e abre o monitor escolhido com Desktop Duplication. O frame chega como textura D3D11. A proteção multithread do dispositivo é ativada para reduzir riscos de deadlock do driver. Mudanças de resolução, HDR, perda do dispositivo ou imagem congelada recriam a sessão. A seleção de monitor e GPU é explícita. Em notebooks híbridos, a saída pertence a um adaptador e o encoder precisa ser validado no mesmo dispositivo para preservar a cadeia sem leitura pela CPU.

Conversão e encode

- O D3D11 Video Processor converte BGRA para NV12 e escala para o perfil solicitado.
- Media Foundation enumera MFTs de hardware ligados ao dispositivo D3D11 escolhido.
- A negociação prefere AV1, depois H.265 e finalmente H.264 quando ambos suportam.
- O encoder solicita baixa latência, GOP de cerca de um segundo, zero B-frames, IDR sob demanda e bitrate dinâmico por ICodecAPI.
- Cada espectador recebe faixa de encode, fila e recuperação próprias.
- O limite de sessões do encoder é sondado e pode reduzir o teto de espectadores.

Perfis adaptativos

Banda aproximada	Perfil de vídeo
0,8 a 1 Mbps	960 x 540 a 30 FPS
Cerca de 2 Mbps	1280 x 720 a 30 FPS
Cerca de 5 Mbps	1280 x 720 a 60 FPS
Cerca de 12 Mbps ou mais	Até 1920 x 1080 a 60 FPS

O controlador trabalha entre aproximadamente 0,8 e 35 Mbps. Perda e RTT derrubam o bitrate rapidamente; a recuperação é gradual para evitar oscilação.

Decodificação e apresentação

No espectador, decoder.rs ativa o decoder Media Foundation na GPU selecionada e entrega superfícies DXGI NV12. presenter.rs apresenta essas superfícies numa swapchain D3D11 flip-discard, com letterbox, resize, fullscreen e reconstrução após device lost. A janela de streaming é nativa e separada do WebView.

A versão 0.7.4 corrigiu a inicialização COM usada pela descoberta WASAPI e pela detecção dos decoders, inclusive quando o comando Tauri já está numa thread STA.

5. Áudio da transmissão

Origem e isolamento

O Voxa usa WASAPI loopback. O host pode capturar todo o dispositivo de saída ou escolher a sessão de áudio de um processo e seus filhos. A captura isolada transmite somente o jogo e impede que Discord e o próprio Voxa retornem ao espectador.

A lista mostra processos com sessão ativa ou pausada no mixer e atualiza a cada dois segundos. A captura isolada requer Windows 10 build 20348 ou superior; sistemas anteriores devem usar o loopback completo.

Formato e sincronização

- PCM float, 48 kHz, dois canais.
- Quadros de 20 ms codificados em Opus.
- Bitrate adaptativo quando a rede fica limitada.
- Áudio e vídeo compartilham a referência de tempo da captura.
- O espectador atrasa áudio adiantado de forma limitada e exibe a diferença A/V.
- A fila de reprodução é limitada para impedir atraso acumulado e crescimento de memória.

O Voxa não captura microfone. A prevenção de eco consiste em excluir Discord e Voxa ao selecionar somente o processo do jogo, não em aplicar cancelamento acústico de microfone.

6. Transporte UDP e redes ruins

Protocolo de datagramas

Elemento	Comportamento
Identificação	Magic VOXA e versão binária 1
Tamanho	Fragmentos para datagramas internos de até 1.178 bytes
Criptografia	ChaCha20-Poly1305 autentica cabeçalho e payload
Chaves	Separadas por direção e por espectador
Replay	Janela antirreplay; reordenação válida é aceita uma vez
Frame	Limite aproximado de 4,4 MiB
Remontagem	Até três frames pendentes; fora de ordem e duplicatas são tratados
Prazos	250 ms para delta e 1,5 s para keyframe

Perda, FEC e keyframes

Frames incompletos ou atrasados são descartados. O Voxa não retransmite vídeo antigo, pois isso aumentaria a latência. Quando a perda quebra a referência, o espectador solicita IDR; o pedido é limitado a quatro por segundo. Keyframes recebem FEC XOR por grupos. A redundância reconstrói um fragmento perdido em cada grupo sem retransmissão. Frames delta priorizam atraso baixo e não recebem o mesmo custo.

Fila, pacing e CPU

- Cada faixa mantém no máximo um frame esperando; sob pressão, o antigo é descartado.
- O pacing impede que todos os fragmentos de um keyframe sejam despejados juntos no roteador.
- Sinais nativos e condition variables acordam workers quando chegam dados, reduzindo CPU ociosa.
- Heartbeats medem RTT, perda e disponibilidade; cada espectador tem métricas próprias.

NAT traversal e relay

O mesmo socket Tokio consulta STUN e faz hole punching. Há fallback entre Cloudflare e dois servidores Google. Pares no mesmo IP público podem usar o endpoint LAN. Para NAT simétrico ou CGNAT, os clientes disputam uma rota por relay UDP.

O relay usa protocolo VRLY, credenciais diferentes por ponta, TTL de 60 segundos, limite de 4.096 sessões, teto de pacotes por IP e fila limitada. Ele encaminha apenas datagramas já cifrados.

7. Segurança e modelo de confiança

Proteções implementadas

Ameaça	Proteção
Roubo da senha no servidor	Senha e segredo Argon2id nunca são enviados
Reutilização de prova	SPAKE2 produz troca efêmera com confirmação mútua
Servidor substituindo chaves	Código E2E de seis dígitos aprovado pelo host
Escuta de mídia	ChaCha20-Poly1305 com chave X25519 desconhecida do servidor
Replay e alteração	Sequência antirreplay e autenticação AEAD
Um viewer lendo outro	Chaves, transportes e relays separados
Flood HTTP e Socket.IO	Rate limits, hello timeout e reserva de capacidade
XSS e conteúdo web	Sem chat, CSP restrita e mídia fora do DOM
Segredos no bundle	Somente a URL do signaling entra no frontend

Identidade persistente

A identidade privada X25519 e até 64 pares confiáveis ficam protegidos no Gerenciador de Credenciais do Windows. O usuário pode limpar a lista pelo painel. O diagnóstico não exporta senha, segredo derivado, chave privada ou chave de mídia.

O pareamento persistente é opcional. A aprovação local continua sendo a fronteira de consentimento: tela e áudio permanecem bloqueados até a autorização.

Segurança do desktop e da entrega

- Tauri usa uma lista mínima de capacidades para janela, updater e reinício.
- A CSP bloqueia objetos, frames e mídia web; conexões ficam restritas a IPC, HTTPS/WSS e localhost de desenvolvimento.
- O manifest Windows não solicita privilégios administrativos.
- Ações de terceiros do GitHub são fixadas por hash de commit.
- Releases são assinadas e o atualizador possui somente a chave pública.

8. Signaling, relay e infraestrutura

Servidor de matchmaking

O servidor Node 24 usa Express 5, Socket.IO 4 e express-rate-limit. As salas vivem somente em memória: um host, até quatro espectadores e nenhum banco. O endpoint /health não revela salas, IPs, rotas ou chaves.

Arquivo	Função
server/index.js	HTTP, Socket.IO, CORS, limites e configuração
handlers.js	Handshake, PAKE opaco, ready, saída e eventos
state.js	Registro de salas, host, viewers e pareamentos
security.js	IP observado, sanitização e rate limiting
admission.js	Controle de conexões antes da autenticação
relay.js	Relay UDP cego e limites de abuso

Implantação gratuita

- Signaling: Render Web Service no plano free, definido em render.yaml.
- Relay: processo Node numa VM com IP público e UDP 3479, previsto para Oracle Cloud Ampere A1 Always Free quando houver capacidade.
- A rota direta continua preferida e não consome banda da VM.
- O free tier do Render pode suspender o serviço e atrasar a primeira conexão.

Variáveis operacionais

Variável	Local	Uso
TRUST_PROXY	Render	Interpreta o IP atrás do proxy
ORIGIN	Render	Origem aceita pelo Socket.IO
VOXA_MAX_VIEWERS	Render	Teto lógico da sala, padrão 4
VOXA_RELAY_PUBLIC_ENDPOINT	Render	IP e porta do relay
VOXA_RELAY_SECRET	Render e VM	Autentica alocações do relay
VITE_SIGNALING_URL	GitHub Actions	Grava a URL HTTPS no painel
TAURI_SIGNING_PRIVATE_KEY	GitHub Actions	Assina artefatos do updater

9. Interface e ciclo de vida

Painel principal

- Alternância entre Hospedar e Conectar.
- Seleção de monitor e GPU do host e GPU do espectador.
- Seleção de todo o áudio ou somente o processo do jogo.
- Código, senha e URL de signaling em painel avançado.
- Aprovação e rejeição de espectadores com código E2E.
- Troca de monitor, áudio e cursor durante a sessão.
- Telemetria de rota, RTT, perda, bitrate, codec, frames, A/V e percentis de latência.
- Exportação de diagnóstico e instalação de update assinado.

Janela de streaming

A janela stream é criada pelo Tauri, mas o vídeo é apresentado diretamente por D3D11 usando o HWND. Fechar essa janela encerra a sessão do viewer; fullscreen é controlado pelo motor nativo. Um Error Boundary impede que uma falha React derrube silenciosamente todo o painel.

Bandeja e memória

Fechar a janela principal a esconde na bandeja para manter o streaming. Ao ocultar, o Voxa chama EmptyWorkingSet somente nos processos WebView2 filhos. O processo Rust e outra instância ativa não são podados, evitando page faults no caminho da mídia.

Atualizador

O painel consulta latest.json no GitHub Releases. Uma atualização é baixada, validada pela chave pública embutida e instalada em modo passivo. A instalação é adiada durante uma transmissão ativa.

10. Diagnóstico e recuperação

Telemetria

Métrica	Finalidade
Rota	Mostra direto, LAN ou relay e o endpoint em uso
RTT e perda	Alimenta o controle de congestionamento
Bitrate de vídeo e áudio	Explica mudanças de qualidade
Frames exibidos e descartados	Indica saúde da remontagem e decoder
Capture, encoder, decoder e audio	Localiza o estágio da falha
P50, P95 e P99	Mostra estabilidade e cauda da latência
Métricas por espectador	Evita sobrescrita pelo último cliente
Capacidade do encoder	Explica o teto real de espectadores

Gravador circular e exportação

O motor mantém no máximo 512 eventos dos últimos 60 segundos. O buffer registra transições e falhas sem crescer indefinidamente. Em caso de crash, outro módulo preserva um log de pânico local.

Exportar diagnóstico gera um JSON em Documentos/Voxa com GPU, driver, monitores, sessões de áudio, codecs, rotas, relay, estados, métricas e eventos. Credenciais e chaves privadas ficam de fora.

Recuperação automática

- Watchdog de cerca de dois segundos para encoder ou captura sem frames.
- Recriação após troca de resolução, HDR, fullscreen, alt-tab ou device lost.
- Pedido de keyframe quando a remontagem perde referência.
- Novo matchmaking quando a interface de rede ou o IP muda.
- Isolamento por espectador: a falha de uma faixa não encerra as demais.

11. Estrutura e dimensão do código

O inventário de `src`, `src-tauri`, `server` e `scripts` soma aproximadamente 10.317 linhas em 54 arquivos de código e automação, sem contar dependências, builds ou ícones.

Camada	Principais arquivos	Papel
Frontend	<code>App.tsx</code> e <code>src/lib</code>	UI, IPC, signaling e estado
Orquestração Rust	<code>native/mod.rs</code> e <code>model.rs</code>	Sessão, peers, comandos e status
Mídia Windows	<code>capture</code> , <code>converter</code> , <code>encoder</code> , <code>decoder</code> , <code>presenter</code> e <code>audio</code>	Pipeline GPU e WASAPI
Transporte	<code>transport.rs</code> , <code>fanout.rs</code> e <code>route.rs</code>	UDP, rotas, filas e feedback
Núcleo puro	<code>native-core/src</code>	Protocolo, AEAD, STUN, FEC e congestionamento
Servidor	<code>server/index.js</code> e <code>server/lib</code>	Matchmaking, segurança e relay
Entrega	<code>scripts</code> e <code>workflows</code>	Validação, empacotamento e promoção

Catálogo dos módulos nativos

Módulo	Responsabilidade
<code>capture.rs</code>	Monitores, adaptadores, driver, DXGI e cursor
<code>converter.rs</code>	Escala e conversão BGRA para NV12 na GPU
<code>encoder.rs</code>	MFTs, codecs, low latency, IDR e bitrate
<code>pipeline.rs</code>	Faixas individuais, perfis e watchdog do host
<code>decoder.rs</code>	Media Foundation e superfícies DXGI
<code>presenter.rs</code>	Swapchain, letterbox, fullscreen e device lost
<code>viewer.rs</code>	Loop do espectador, A/V, latência e GPU
<code>audio.rs</code>	WASAPI, sessões por processo, Opus e playback
<code>transport.rs</code>	Socket, cifra, fragmentos, feedback e workers
<code>pake.rs</code> e <code>identity.rs</code>	SPAKE2, X25519 e credenciais Windows
<code>diagnostics.rs</code> e <code>com.rs</code>	Eventos circulares e inicialização COM

Dependências centrais

Ecosistema	Dependências
Frontend	React 19, Tauri API, updater, process e Socket.IO client
Rust app	Tauri 2, Tokio, windows, rusty-opus, Argon2, Pakery SPAKE2 e keyring
Rust core	ChaCha20-Poly1305, X25519, SHA-256 e Tokio
Servidor	Node 24, Express 5, Socket.IO 4 e express-rate-limit

12. Testes, CI e qualidade

Cobertura automatizada

- Testes Node para admissão, limites, sanitização, quatro viewers, PAKE e relay.
- Testes Rust para cifra, adulteração, replay, fragmentação, chaves e codecs.
- Testes de remontagem, deadlines, limites de memória e recuperação FEC.
- Simulações determinísticas de jitter, perda em rajada e congestionamento.
- Testes de contrato Rust/React e validações do PE e manifest do instalador.
- Fuzzing semanal do protocolo cifrado e da remontagem/FEC.

Integração contínua

- 1 NPM audit do app e servidor e RustSec sobre Cargo.lock.
- 2 TypeScript, build Vite e suíte Node.
- 3 Rustfmt, Clippy com warnings como erro e testes Rust no Windows 2025.
- 4 Dependabot semanal para npm, Cargo e GitHub Actions.

Pipeline de release

- 1 Confere que a tag corresponde à versão do projeto.
- 2 Executa novamente todo o CI.
- 3 Compila o Tauri e gera NSIS, MSI, assinaturas e latest.json.
- 4 Valida dependências e manifest do executável.
- 5 Instala, atualiza e desinstala numa VM Windows limpa.
- 6 Somente após sucesso promove a release de draft para pública.

Estado da versão 0.7.4

A release documentada possui EXE, MSI, assinaturas e latest.json. Os workflows de validação, build, dependências, instalação, atualização e desinstalação concluíram com sucesso. Durante a correção desta versão, passaram 33 testes do app e servidor e 23 testes do núcleo UDP/FEC.

Checks automatizados comprovam compilação e contratos testados. Eles não substituem sessões prolongadas entre dois PCs e diferentes famílias de GPU.

13. Como usar e operar

Para hospedar

- 1 Abra o Voxa, escolha Hospedar e selecione monitor e GPU.
- 2 Escolha todo o áudio ou abra o jogo, reproduza som e selecione o processo.
- 3 Defina o mesmo código e senha forte usados pelo espectador.
- 4 Comece a transmissão e aguarde o pedido de conexão.
- 5 Compare o código E2E e aprove o computador correto.
- 6 Acompanhe rota, perda, bitrate, frames e métricas por espectador.

Para assistir

- 1 Escolha Conectar e selecione a GPU ou mantenha Automática.
- 2 Use o mesmo código e senha da sala.
- 3 Conecte e confira se o código E2E coincide com o do host.
- 4 Após aprovação, aguarde a janela Voxa Stream receber o primeiro keyframe.

Solução de problemas

Sintoma	Primeira verificação
Servidor não respondeu	Aguarde o cold start do Render e tente de novo
Nenhum codec compatível	Atualize Voxa e driver; teste GPU automática
Lista de áudio vazia	Faça o jogo produzir som e atualize processos
Tela branca	Exporte diagnóstico e confira decoder e frames
Só conecta em algumas redes	Confirme o relay UDP 3479
Imagem degrada	Observe perda, RTT e perfil adaptativo

14. Desenvolvimento e implantação

Ambiente local

O fluxo principal é instalar as dependências do app e servidor, rodar `npm run verify`, compilar o frontend, executar os testes Rust e passar o Clippy com warnings tratados como erro.

No Windows GNU, um `target-dir` sem Unicode evita falhas de ferramentas antigas com caminhos como Tudão. O CI usa MSVC e Windows 2025 e é a referência para testes que ligam Tauri e WebView2.

Signaling no Render

`render.yaml` cria o serviço `voxa-signaling` no plano free, instala dependências de produção, executa `server/index.js` e monitora `/health`. Depois de mudar servidor ou variáveis, `scripts/check-deployment.mjs` valida a resposta pública.

Relay na Oracle

O relay inicia com `npm run relay` no servidor, escuta UDP 3479 e exige o mesmo segredo do Render. O firewall da nuvem e do sistema deve expor somente a porta necessária. O uso gratuito depende dos limites e da disponibilidade da conta Oracle Cloud.

Versionamento

A versão precisa coincidir em `package.json`, `package-lock.json`, `Cargo.toml`, `Cargo.lock`, `native-core` e `tauri.conf.json`. Enviar uma tag `vX.Y.Z` inicia a release, e a promoção bloqueia instaladores que falhem nos testes.

15. Limitações e riscos conhecidos

Tema	Situação atual e impacto
Validação física	Falta ciclo formal de 30 min entre PCs e NVENC, AMF e QuickSync
Relay gratuito	Uma região aumenta latência e risco de indisponibilidade
Render free	Cold start pode atrasar a primeira conexão
GPU e driver	MFTs variam por fabricante e versão
Encoders simultâneos	Licenças e drivers podem limitar sessões sob carga
Áudio por processo	Requer Windows 10 build 20348 ou superior
HDR e fullscreen	Combinações de jogo e driver ainda precisam de matriz real
IPv6	STUN e relay atuais priorizam IPv4
Controle remoto	Não existe na linha atual do produto

Critério recomendado de prontidão

- Sessões de 30 minutos em 1080p60 entre redes distintas.
- Cenas rápidas, alt-tab, mudança de resolução, HDR e fullscreen.
- Perfis de 0, 1, 3, 5 e 10 por cento de perda, com jitter e limite de banda.
- P50, P95 e P99 de captura até tela, estabilidade A/V e memória constante.
- Matriz separada para NVENC, AMF, QuickSync e notebooks híbridos.

16. Próximos passos recomendados

Prioridade 1 - validar o produto

- 1 Executar a matriz física de duas máquinas e guardar diagnósticos por GPU.
- 2 Medir encode, decode, intervalo entre frames e fila do socket.
- 3 Criar um teste guiado antes de hospedar: monitor, encoder, áudio, STUN, UDP e relay.

Prioridade 2 - resiliência operacional

- 1 Validar ou provisionar um segundo relay gratuito em outra região quando houver capacidade sem custo.
- 2 Adicionar IPv6 e dual stack com verificação segura do endpoint.
- 3 Automatizar reconexão com troca de interface durante vídeo e áudio.

Prioridade 3 - manutenção

- 1 Continuar dividindo transport.rs e native/mod.rs, os maiores módulos.
- 2 Atualizar dependências somente após CI, RustSec e teste de instalador.
- 3 Manter diagnósticos sem segredos e ampliar eventos apenas quando forem úteis.

17. Glossário

Termo	Definição
DXGI Desktop Duplication	Entrega frames do desktop como texturas D3D11
D3D11	API gráfica para texturas, conversão, apresentação e swapchain
Media Foundation e MFT	Framework e transformações de codec do Windows
WASAPI	Captura loopback e reprodução de áudio no Windows
STUN	Descobre endereço e porta públicos
Hole punching	Cria rota direta UDP através do NAT
Relay cego	Encaminha pacotes cifrados sem conhecer conteúdo ou chave
FEC	Redundância para recuperar perda sem retransmissão
IDR ou keyframe	Frame completo que reinicia a referência do decoder
AEAD	Criptografia com confidencialidade e autenticação
X25519	Troca de chaves entre os computadores
SPAKE2	PAKE que confirma a senha sem transmiti-la
Argon2id	Derivação resistente a tentativas massivas
AIMD	Controle que reduz rápido e recupera gradualmente

18. Referências internas e conclusão

Este relatório foi consolidado diretamente a partir dos seguintes grupos da versão 0.7.4:

- README.md e docs/NATIVE-STREAMING-ROADMAP.md.
- src/App.tsx e src/lib para painel, signaling e IPC.
- src-tauri/src/native e src-tauri/native-core para mídia, rede e segurança.
- server/index.js e server/lib para matchmaking, limites e relay.
- render.yaml, tauri.conf.json e docs/DEPLOY.md para infraestrutura e distribuição.
- .github/workflows e dependabot.yml para CI, segurança, fuzzing e releases.

Repositório: github.com/AntonyAraujo87/Voxa. Release analisada: Voxa 0.7.4.

Conclusão

O Voxa 0.7.4 é uma base funcional e tecnicamente ambiciosa de streaming nativo para Windows. A arquitetura retira a mídia do navegador, mantém frames na GPU, isola espectadores e usa criptografia ponta a ponta independente do servidor. O próximo salto de qualidade depende principalmente de validação física sistemática, medições do caminho completo e diversidade de redes e GPUs, não de novos recursos sociais.